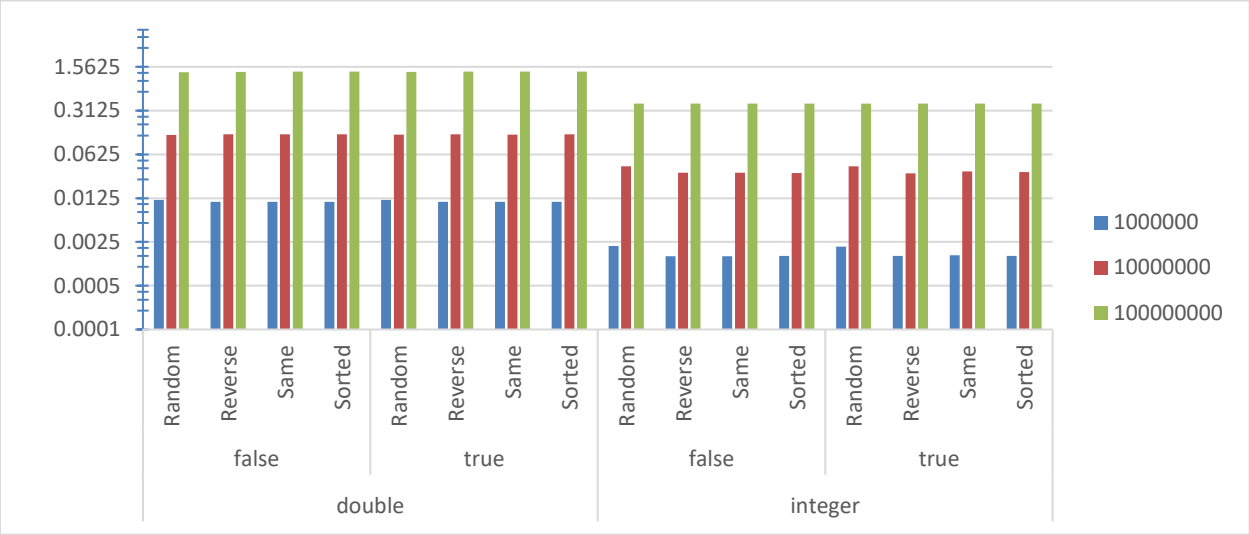
part 0 – setup:

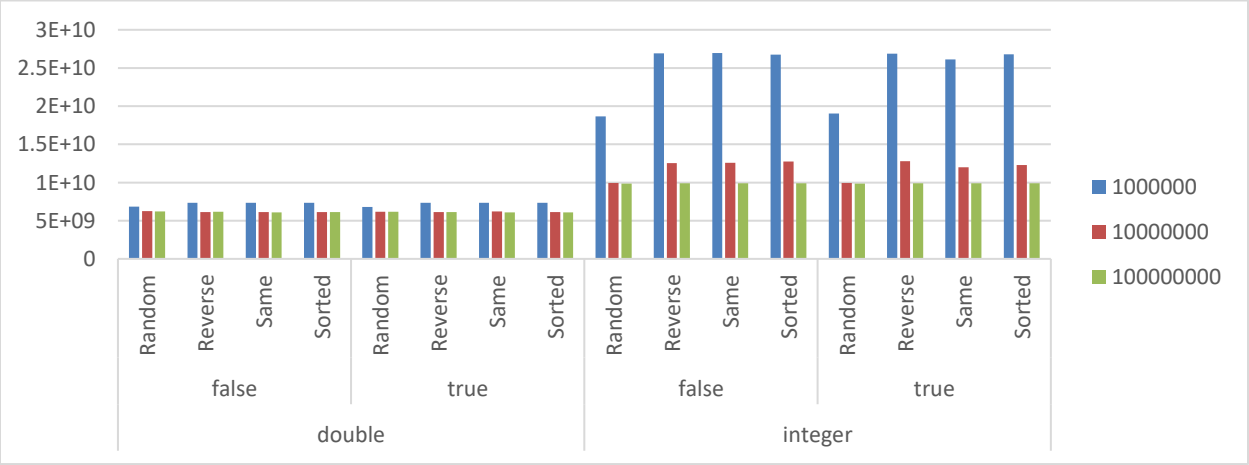
The code for this project was created in C++, using no external libraries (apart from the C++ standard library.) The input `.mtx` files for part 5 were generated using Python's `networkx` library.

All setups were compiled using G++, with the compiler flags `-std=c++17 -Isrc -MMD -MP -O3`. That is, except for part 2, which swaps out the compiler optimizations as part of the testing process.

Each test kernel is run once to 'warm-up' the machine, followed by ten iterations to measure the total performance.

part 1 – array maximum:

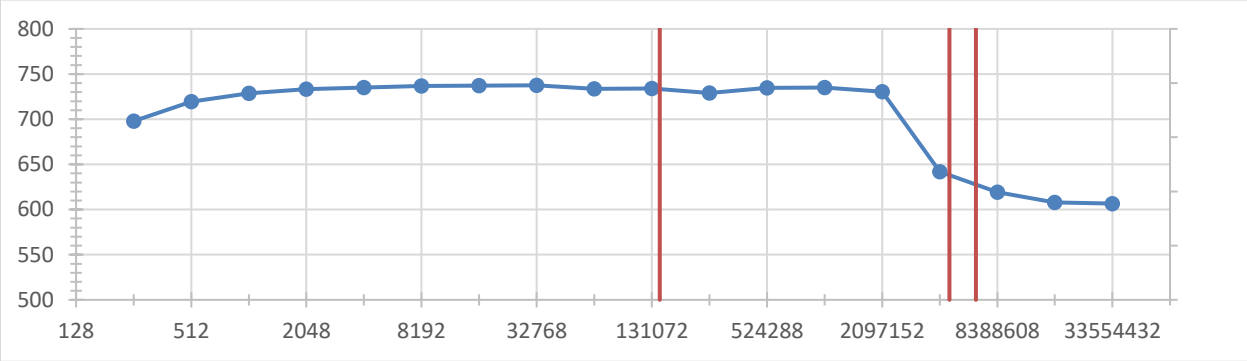




The first chart depicts the total runtime for each variation of the algorithm on each input size. The X-axis is each variation of the algorithm (double/integer input array, branchless = true / false, array fill type) Each bar is a different *n* input size, and the Y-axis is in seconds, scaled logarithmically.

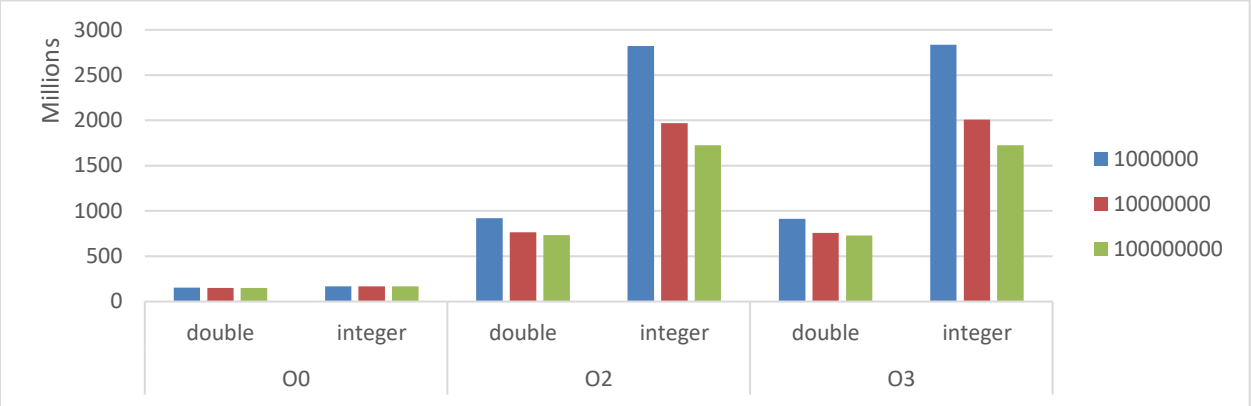
The second chart depicts the bytes per second, or memory bandwidth, of the `arrayMax` algorithm. The X-axis is the same as the previous chart – each variation of the algorithm, for each given *n* – while the Y-axis holds the bytes per second.

As you can see, the branchless / branch-full versions make no impact on the performance – I suspect it is completely optimized away by the compiler. Interestingly, the double / integer versions have very different performance characteristics, particularly in relation to the array sort type! For doubles, each array sort type ran at roughly the same speed, though for the integers, the "random" array sort type was a fair bit slower than the other options. I wonder if this, too, is due to compiler optimizations? Very interesting, nonetheless.



This chart depicts the bandwidth, along the Y-axis (in giga-bytes / second), with the number of doubles processed along the X-axis, scaled logarithmically. Marked in red is the size of each cache. While the performance impact of running out of L1 cache is negligible, the performance decline after the end of the L2 / L3 caches is *dramatic*!

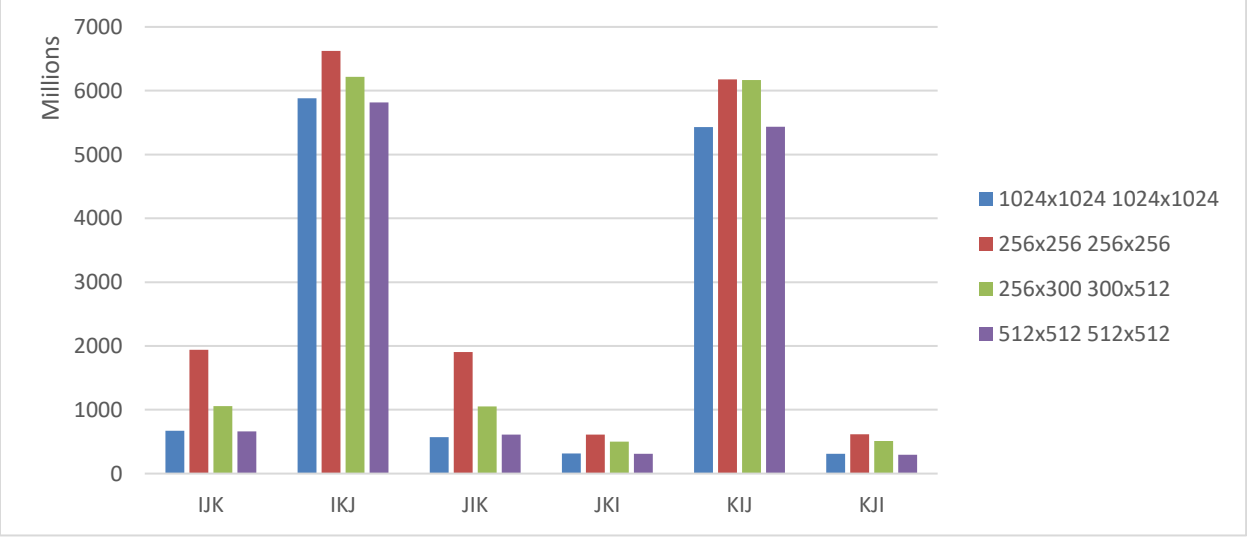
part 2 – prefix sum:



The Y-axis represents the elements processed per second, in millions, and the X-axis is the optimization level, element type (double vs. integer) and input size *n*.

As expected, the O0 variant has the lowest throughput, with the O2 and O3 results being roughly equivalent. Lower input size also results in higher throughput.

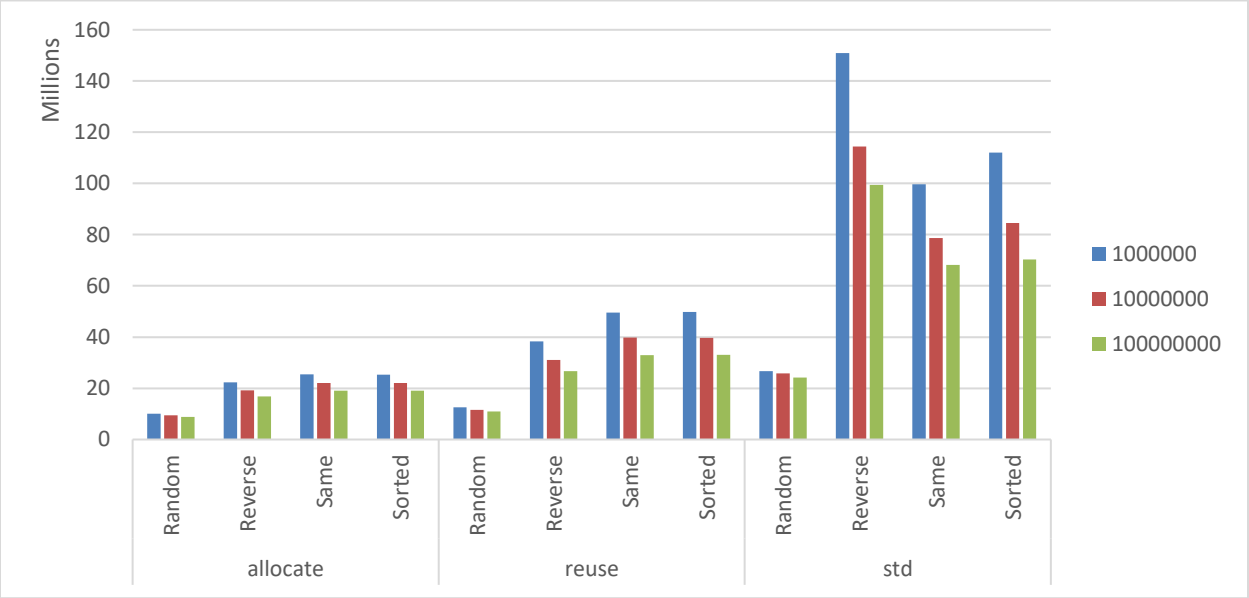
part 3 – matrix multiplication:



The X-axis is the iteration order + matrix size, while the Y-axis is the FLOPs, in millions.

As you can see, the order of iteration has a major impact on performance! The IKJ and KIJ systems are especially slow, while the JKI and KJI systems are comparatively lightning-fast. It seems the algorithms that iterate `j` the most frequently are the slowest, while the ones that iterate `i` the most frequently are the quickest. This makes sense, given the index order or the matrices – values of `i` within the matrix are nearby in memory, whereas different values of `j` are as far apart as possible in memory, leading to cache thrashing!

part 4 – merge sort:

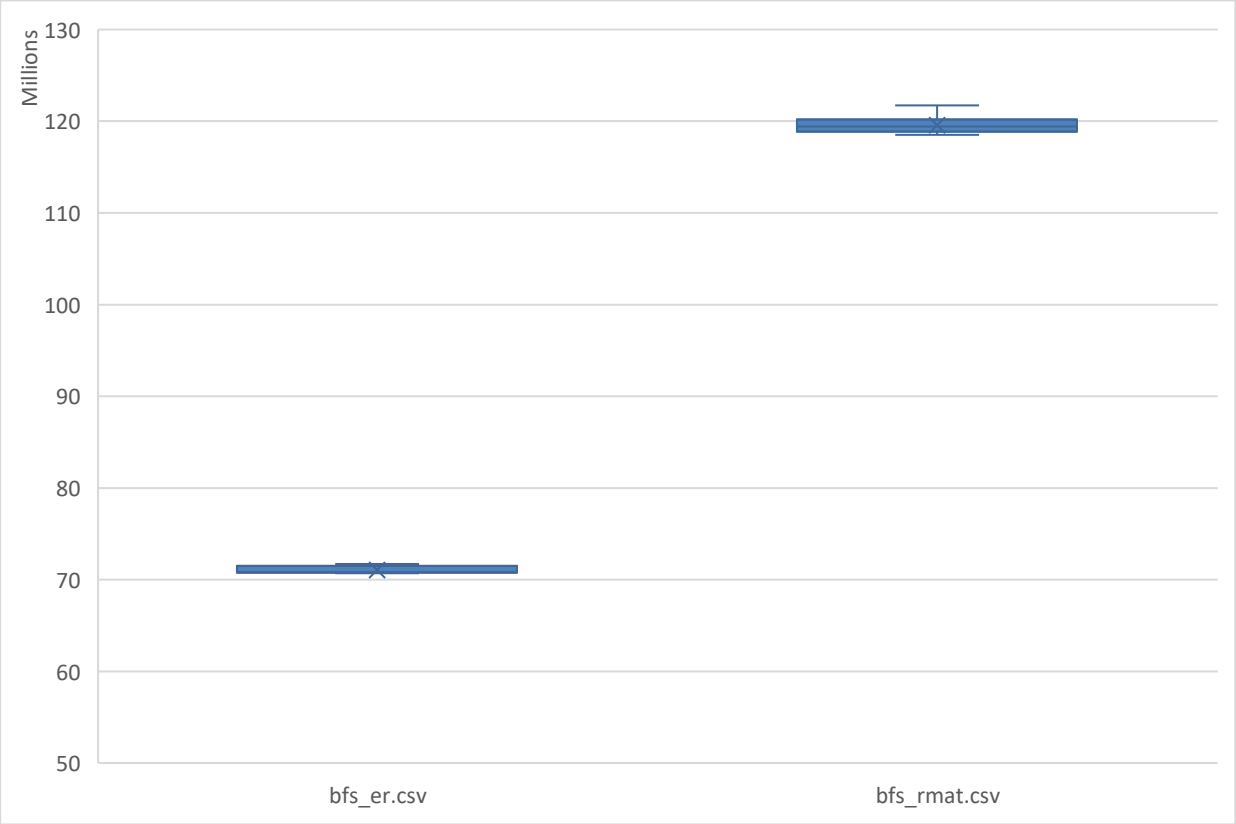


The X-axis of this chart represents the algorithm variation + array type + input size, while the Y-axis is the elements-per-second, in millions.

The speeds for each algorithm are as expected – the algorithm from the C++ standard library is far-and-away the fastest compared to my humble ad-hoc merge sort, and the allocate-per-merge algorithm is slower than the reuse variant, though not significantly so.

The array arrangements, on the other hand, are more interesting. The random arrangement is the slowest for all three algorithms, with the C++ standard implementation taking the most significant performance hit. Interestingly, the reversed array seems to perform significantly better in C++'s implementation, while being the slowest case in the naïve merge sort. The same and sorted arrays appear to perform roughly equally in all three versions.

part 5 – graph traversal:



This graph compares the edges / second (in millions, along the Y axis) between the two graph formats. Though it may be difficult to tell, it is reporting the minimum / maximum / median between all 16 runs of the profiling – the results were just largely consistent between them.

As you can see, the RMAT graph structure tended to be traversed faster than the Erdos-Reniy structure – about 1.7x times faster, approximately.

The following two pages are tables that show some information about each individual BFS run, for each graph. Each row represents a different search. It's necessarily quite compressed - I've had to frame them horizontally, such that the information actually fits – but here they are:

erdos-reniy graph:

0		1		2		3		4		5		6		7	
Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached
16	0.00002	246	0.00025	3943	0.00401	60333	0.06155	592409	0.62651	391576	0.99995	52	1	0	1
20	0.00002	321	0.00033	5278	0.00536	80673	0.08229	681791	0.7325	280485	0.99999	7	1	0	1
7	0.00001	110	0.00011	1729	0.00176	27257	0.02775	347199	0.35887	668945	0.99683	3328	1	0	1
18	0.00002	306	0.00031	4799	0.00489	73076	0.07458	651952	0.69633	318408	0.99998	16	1	0	1
12	0.00001	192	0.00019	3001	0.00306	46813	0.0477	509820	0.5339	488532	0.9998	205	1	0	1
10	0.00001	190	0.00019	2929	0.00298	45622	0.04649	501180	0.52446	498411	0.99978	233	1	0	1
16	0.00002	269	0.00027	4270	0.00434	66032	0.06732	620873	0.65943	357092	0.99998	23	1	0	1
14	0.00001	225	0.00023	3666	0.00372	57097	0.05818	574022	0.60561	413487	0.99994	64	1	0	1
16	0.00002	257	0.00026	4066	0.00414	62848	0.06407	604778	0.64084	376572	0.99996	38	1	0	1
19	0.00002	296	0.0003	4654	0.00474	71400	0.07283	644799	0.68776	327380	0.99997	27	1	0	1
14	0.00001	238	0.00024	3832	0.00389	59568	0.0607	588494	0.62193	396374	0.99995	55	1	0	1
17	0.00002	262	0.00027	4145	0.00422	64377	0.06561	612814	0.65004	366934	0.99997	26	1	0	1
14	0.00001	231	0.00023	3681	0.00374	57339	0.05843	575095	0.60688	412158	0.99994	57	1	0	1
17	0.00002	285	0.00029	4517	0.0046	70087	0.07144	638847	0.68069	334797	0.99998	25	1	0	1
25	0.00002	402	0.00041	6495	0.0066	98628	0.10066	733414	0.8001	209606	0.99999	5	1	0	1
15	0.00001	276	0.00028	4450	0.00452	68659	0.07	633590	0.67424	341569	0.99998	16	1	0	1

RMAT graph:

0		1		2		3		4		5		6		7		8		9	
Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached	Frontier Size	% Reached
1	0	12	0.00001	5580	0.00533	277597	0.27007	419360	0.67	137218	0.80087	3100	0.80382	53	0.80387	1	0.80387	0	0.80387
3	0	3158	0.00301	236936	0.22897	424634	0.63394	173460	0.79936	4654	0.8038	76	0.80387	1	0.80387	0	0.80387		
1	0	3205	0.00306	236263	0.22838	424773	0.63347	173913	0.79933	4688	0.8038	77	0.80387	2	0.80387	0	0.80387		
338	0.00032	101080	0.09672	419262	0.49656	303985	0.78646	17961	0.80359	291	0.80387	5	0.80387	0	0.80387				
30	0.00003	19768	0.01888	354173	0.35665	397040	0.73529	70633	0.80265	1258	0.80385	20	0.80387	0	0.80387				
3	0	8893	0.00848	305720	0.30004	414318	0.69517	111679	0.80167	2283	0.80385	26	0.80387	0	0.80387				
2	0	324	0.00031	89085	0.08527	502869	0.56484	242084	0.79571	8420	0.80374	138	0.80387	0	0.80387				
13	0.00001	4235	0.00405	258265	0.25035	423393	0.65413	153217	0.80025	3747	0.80382	52	0.80387	0	0.80387				
3	0	3139	0.003	236063	0.22812	425399	0.63382	173527	0.7993	4719	0.8038	72	0.80387	0	0.80387				
3	0	1235	0.00118	175459	0.16851	426722	0.57547	231030	0.79579	8331	0.80374	137	0.80387	5	0.80387	0	0.80387		
3	0	401	0.00039	102748	0.09837	419878	0.4988	302027	0.78684	17569	0.80359	289	0.80387	7	0.80387	0	0.80387		
10	0.00001	10632	0.01015	315529	0.31106	410386	0.70244	104262	0.80187	2061	0.80383	41	0.80387	1	0.80387	0	0.80387		
5	0	2109	0.00202	213015	0.20516	426275	0.61169	195470	0.79811	5956	0.80379	92	0.80387	0	0.80387				
25	0.00002	21522	0.02055	358255	0.36221	393598	0.73757	68316	0.80272	1178	0.80385	28	0.80387	0	0.80387				
4	0	4118	0.00393	257397	0.2494	422448	0.65228	155074	0.80017	3819	0.80381	60	0.80387	2	0.80387	0	0.80387		
3	0	4314	0.00412	399688	0.38529	424514	0.79014	14182	0.80366	218	0.80387	3	0.80387	0	0.80387				